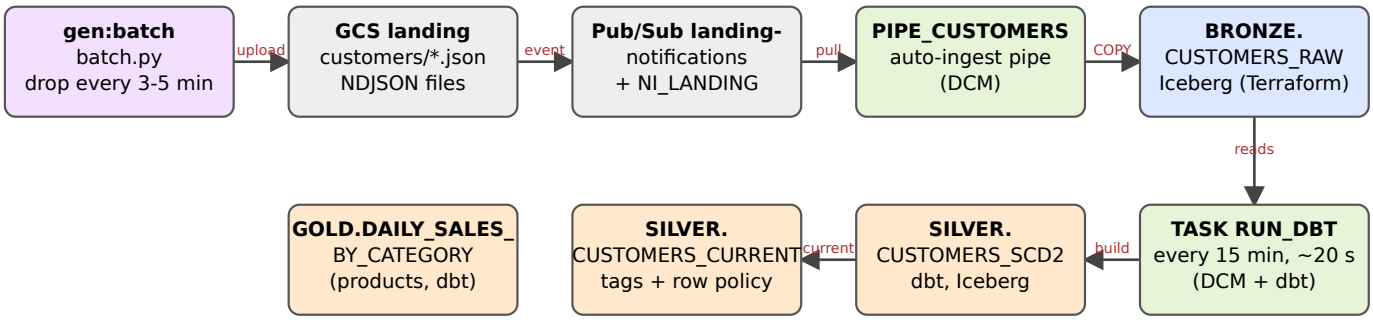
Lineage report: team 06 (TBD lakehouse, 2026z)

Project `tbd-2026z-stud06-data`, Snowflake account on Google Cloud Europe West 4. One record is followed from its generator to the last table on each ingestion path. Timings are measured from the live system on 2026-11-11 (Snowflake `COPY_HISTORY`, `TASK_HISTORY`, `DYNAMIC_TABLE_REFRESH_HISTORY`, `GET_LINEAGE`, `TAG_REFERENCES_ALL_COLUMNS`); the steps outside Snowflake (GCS, Pub/Sub, bridge) are taken from the code.

1. Batch path: a customer change (file) to silver, and the gold built from the batch files



Colour = tool that created the object: Terraform | DCM | dbt | Python in src/ | GCP resource (Terraform, gcp/)

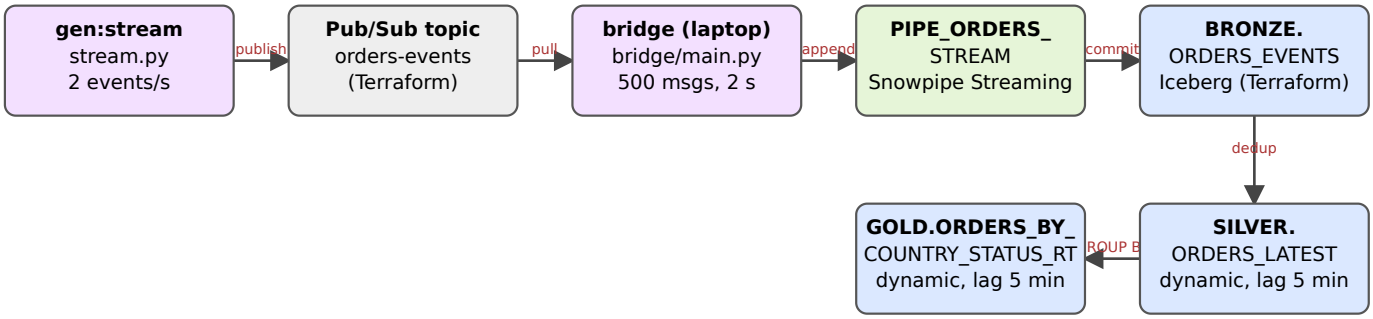
Finding: no gold table is built from customer files. `SNOWFLAKE.CORE.GET_LINEAGE` upstream of `GOLD.CUSTOMER_LTV` returns only `BRONZE.ORDERS_EVENTS` (distance 1): the model hashes `customer_id` from the order events (`dbt/models/gold/customer_ltv.sql`) and never joins `CUSTOMERS_CURRENT`. So a customer change from `gen:batch` ends in `SILVER.CUSTOMERS_CURRENT` (the PII table); the batch records that reach gold are products and returns (`DAILY_SALES_BY_CATEGORY`, `RETURN_RATES`, upstream: `PRODUCTS_RAW` -> `PRODUCTS`, `RETURNS_RAW` -> `RETURNS`). `GOLD.CUSTOMER_LTV` is built by the batch tool (dbt task) but fed by the streaming bronze table.

#	Object	What moves the data in	Created by / file	Time	Format	Readable by
1	**gen:batch** (`mise run gen:batch`)	Python process on the laptop; every 180-300 s (random) it writes up to 20 changed/new customers, 5 products, up to 10 returns	Python: `src/tbd_lakehouse/generators/batch.py`, `model.py`	3-5 min between drops	JSON lines (not Iceberg)	n/a
2	GCS bucket `tbd-2026z-stud06-data-landing`, path `customers/customers-<stamp>-<n>.json`	generator uploads the object	Terraform: `gcp/storage.tf`	upload: seconds	NDJSON files, no table format	anything with bucket IAM
3	GCS notification -> Pub/Sub topic `landing-notifications`, subscription `landing-notifications-snowpipe`	GCS event OBJECT_FINALIZE publishes a message per file	Terraform: `gcp/storage.tf` (`google_storage_notification`), `gcp/pubsub.tf`	seconds	messages	Snowflake via `NI_LANDING` (`snowflake/integrations.tf`)
4	Stage `BRONZE.LANDING_STAGE` + `PIPE_CUSTOMERS` (AUTO_INGEST)	Snowpipe reads the notification and runs `COPY INTO BRONZE.CUSTOMERS_RAW`	DCM: `dcm/sources/definitions/ingest.sql` (integrations: Terraform)	**measured 19 s - 2 min 45 s** from the drop time in the file name to `LAST_LOAD_TIME` in COPY_HISTORY (e.g. 19:13:35 -> 19:15:53)	pipe is not a table	-
5	`BRONZE.CUSTOMERS_RAW` (all versions, duplicates kept)	Snowpipe writes Parquet + Iceberg metadata to `gs://...-lakehouse/iceberg/bronze/customers_raw`	Terraform: `snowflake/bronze.tf` (`snowflake_iceberg_table.bronze`); PII tags: same file	-	**Iceberg** (Snowflake-managed, catalog SNOWFLAKE, external volume EV_LAKEHOUSE)	Snowflake; Spark via Horizon REST (masked); DuckDB refused (PII table)
6	Task `OPS.RUN_DBT` running `EXECUTE DBT PROJECT ... 'build'`	schedule triggers dbt on `WH_TRANSFORM`	DCM: `dcm/sources/definitions/dbt_task.sql`; project deployed by `mise run dbt:deploy`	**every 15 min**, run takes 19-25 s (TASK_HISTORY)	-	-
7	`SILVER.CUSTOMERS_SCD2` (one row per customer version)	dbt table model reads `BRONZE.CUSTOMERS_RAW`, drops re-sent versions	dbt: `dbt/models/silver/customers_scd2.sql`	within the dbt run (seconds)	Iceberg (`+table_format: iceberg`, `dbt_project.yml`)	Snowflake; Spark (masked)

8	`SILVER.CUSTOMERS_CURRENT` (`is_current` rows)	dbt table model on SCD2; post-hooks tag PII and attach the row access policy	dbt: `dbt/models/silver/customers_current.sql`, `_silver.yml`, macros `apply_pii_tags.sql`, `attach_row_access.sql`	same dbt run	Iceberg	Snowflake; Spark via Horizon (masked + filtered); DuckDB refused
9	**Gold from batch files:** `GOLD.DAILY_SALES_BY_CATEGORY`, `GOLD.RETURN_RATES` (via `SILVER.PRODUCTS/RETURNS`)	same dbt run: product/return change -> `PRODUCTS_RAW/RETURNS_RAW` -> silver -> gold	dbt: `dbt/models/gold/*.sql`, `silver/products.sql`, `returns.sql`	same dbt run	Iceberg, no PII	Snowflake, Spark, DuckDB (PII-free)

End to end (batch): file in bucket -> bronze: 0.3-3 min; bronze -> silver/gold: waits for the next 15-minute dbt run (+20 s). A change is visible in silver **0-18 min** after the drop.

2. Streaming path: an order event to gold



Colour = tool that created the object: Terraform | DCM | dbt | Python in src/ | GCP resource (Terraform, gcp/)

#	Object	What moves the data in	Created by / file	Time	Format	Readable by
1	**gen:stream** (`mise run gen:stream`)	Python process; 2 events/s; every 5th tick an order lifecycle (placed -> paid -> shipped -> delivered or cancelled), otherwise a click	Python: `generators/stream.py`, `model.py`	~2 events/s	JSON messages	n/a
2	Pub/Sub topic `orders-events`, subscription `orders-events-bridge`	publisher client	Terraform: `gcp/pubsub.tf`	ms	messages (retained 7 days)	-
3	**bridge** (`mise run bridge`)	pulls up to 500 messages, appends to a Snowpipe Streaming channel (flush every 2 s), acks only after Snowflake commits: **at-least-once**	Python: `src/tbd_lakehouse/bridge/main.py`, `mapping.py`	flush interval 2 s	-	-
4	`PIPE_ORDERS_STREAM` (Snowpipe Streaming, high-performance)	receives channel rows, `COPY INTO BRONZE.ORDERS_EVENTS ... DATA_SOURCE(TYPE=>'STREAMING')`	DCM: `dcm/sources/definitions/streaming_pipes.sql`	**seconds**: `_INGESTED_AT` is within ~2 s of `event_ts` (median -2.2 s: WSL clock vs Snowflake clock skew, so the true latency is below the skew)	-	-
5	`BRONZE.ORDERS_EVENTS` (duplicates kept)	Snowpipe Streaming commits Parquet to the lakehouse bucket	Terraform: `snowflake/bronze.tf`	-	**Iceberg**	Snowflake; Spark via Horizon (masked); DuckDB refused
6	`SILVER.ORDERS_LATEST` (dynamic Iceberg table)	automatic refresh; removes duplicates by `event_id` and keeps the latest status per order	Terraform: `snowflake/dynamic_tables.tf` (`snowflake_execute.dynamic_table`)	**TARGET_LAG 5 min**; measured refresh every ~3-5 min, each 1-2 s (FULL refresh mode) on WH_TRANSFORM	Iceberg (dynamic)	Snowflake; Spark (masked); DuckDB refused
7	`GOLD.ORDERS_BY_COUNTRY_STATUS_RT` (dynamic Iceberg table)	automatic refresh: `GROUP BY shipping_country, status` over silver	Terraform: `snowflake/dynamic_tables.tf` (`dynamic_table_gold`)	TARGET_LAG 5 min	Iceberg (dynamic), no PII	Snowflake, Spark, DuckDB

End to end (streaming): bronze within seconds; silver/gold follow the 5-minute target lag (silver refreshed every 3-5 min in the history; gold refreshes from silver). An event is in gold within about **5-10 minutes**, versus up to ~18 min for batch.

Lineage as seen by Snowflake (`GET_LINEAGE ... 'UPSTREAM'`): `GOLD.ORDERS_BY_COUNTRY_STATUS_RT` ← `SILVER.ORDERS_LATEST` (distance 1) ← `BRONZE.ORDERS_EVENTS` (distance 2). Snowflake does not see GCS, Pub/Sub or the bridge.

3. Following one PII column on each path

Batch: `email`

Object	Tag PII_TYPE	apply_method	Where it is set / masked
`BRONZE.CUSTOMERS_RAW.EMAIL`	EMAIL	**MANUAL**	`snowflake_tag_association.pii` in `snowflake/bronze.tf` (Terraform). Masking applies here too (bronze is readable but masked).

SILVER.CUSTOMERS_SCD2.EMAIL	EMAIL	**MANUAL**	dbt: meta: {pii_type: EMAIL} in silver/_silver.yml; post-hook macro apply_pii_tags runs ALTER ICEBERG TABLE ... SET TAG after every build (tables are recreated)
SILVER.CUSTOMERS_CURRENT.EMAIL	EMAIL	**MANUAL**	same macro. Row access policy RAP_SUPPORT_COUNTRY is attached on country by attach_row_access (policy defined in snowflake/governance.tf)

Masking is not stored in the data: the policies MP_PII_STRING / MP_PII_DATE are bound to the tag (ALTER TAG PII_TYPE SET MASKING POLICY, governance.tf) and evaluated on every read by CURRENT_ROLE(): PII_ADMIN/PIPELINE_ADMIN see the raw email, SUPPORT ***@example.pl, MARKETING a 12-hex pseudonym, ANALYST ***. PIPELINE_ADMIN owns dbt and the dynamic tables and is exempt, so derived tables hold real values and masking happens at read time. Spark reads the same policies through Horizon's scan-plan API.

Streaming: ip_address

Object	Tag PII_TYPE	apply_method	Where it is set / masked
BRONZE.ORDERS_EVENTS.IP_ADDRESS	IP	**MANUAL**	snowflake_tag_association.pii in bronze.tf (Terraform)
SILVER.ORDERS_LATEST.IP_ADDRESS	IP	**PROPAGATED**	nobody tags it: the tag is created with PROPAGATE = ON_DEPENDENCY_AND_DATA_MOVEMENT (governance.tf) and follows the dynamic table's lineage from bronze; the masking follows the tag
GOLD.ORDERS_BY_COUNTRY_STATUS_RT	-	-	the column does not exist: gold aggregates by country and status, so it is PII-free and DuckDB may read it

Verified with TAG_REFERENCES_ALL_COLUMNS: bronze columns CARD_LAST4, IP_ADDRESS, SHIPPING_STREET, SHIPPING_POSTAL_CODE are MANUAL, the same four columns in SILVER.ORDERS_LATEST are PROPAGATED. Policy by role: SUPPORT 10.20.x.x, MARKETING 10.20.0.0, ANALYST ***.

4. Comparison

	Batch (files -> Snowpipe -> dbt)	Streaming (Pub/Sub -> Snowpipe Streaming -> dynamic tables)
Latency	bronze 19 s-2.8 min after the drop; silver/gold wait for the 15-min dbt schedule: up to ~18 min	bronze in seconds; silver/gold within the 5-min target lag: ~5-10 min
Cost	Snowpipe auto-ingest is serverless (~0.0037 credits/GB, ~0 here: 750 KB in 6 h). dbt: one X-Small run of ~20 s every 15 min (4 runs/h; a resume costs at least 60 s billed, so about 0.07 credit/h at 1 credit/h)	Snowpipe Streaming serverless (~0 at this volume). Dynamic tables use WH_TRANSFORM: each refresh is only 1-2 s but there are many per hour (FULL refresh mode here), and a lower target lag multiplies this (1 min ~ 24 credits/day). The bridge and generators run on the laptop (no Snowflake cost)
Measured today	all warehouses together 0.99 credits (also the tests and my queries); serverless (pipes, streaming, governance, tasks) < 0.01 credits. Budget: 12 credits	
Duplicates	bronze keeps them (2260 rows for 2062 customers). **dbt** customers_scd2 removes re-sent versions: QUALIFY row_number() over (partition by customer_id, updated_at order by _ingested_at desc) = 1 (2260 -> 2240 rows); latest_orders() does the same for orders by event_id	bronze keeps them (2033 events, 2022 distinct event_id: the bridge is at-least-once and Pub/Sub may redeliver). The **dynamic table** ORDERS_LATEST removes them: ROW_NUMBER() OVER (PARTITION BY event_id ORDER BY _ingested_at) = 1, then keeps the latest status per order_id

Both paths land in the same bronze layer in Iceberg on the same bucket; the difference is who does the transformation (a scheduled dbt task vs. declarative dynamic tables) and so who sets the latency/cost trade-off.